

# 신봉고 급식 알리미 프로젝트 작업계획서

## 1. 프로젝트 개요

### 프로젝트명

신봉고 급식 알리미

### 기간

약 3주

### 인원

4명

### 목표

신봉고등학교의 급식 정보를 NEIS에서 가져와, 사용자가 카카오톡에서

오늘 급식

이라고 입력하면 당일 급식 메뉴를 알려주는 챗봇을 만든다.

예상 결과:

사용자:

오늘 급식

급식 알리미:

🍲 8월 27일 신봉고 급식

현미밥

김치찌개

치킨까스

계란말이

깍두기

735 Kcal

이 프로젝트의 가장 중요한 목표는 복잡한 프로그램을 만드는 것이 아니다.

하나의 서비스를 네 개의 작은 프로그램으로 나누고, 네 명이 각각 한 부분을 직접 개발한 뒤 마지막에 하나의 서비스로 연결해보는 것이 핵심이다.

---

## 2. 우리가 이 프로젝트에서 경험할 것

이 프로젝트를 통해 다음을 직접 경험한다.

- Python으로 실제 프로그램 만들기
- 인터넷 API를 이용해 데이터 가져오기
- JSON 데이터 다루기
- 함수로 프로그램 나누기
- 프로그램끼리 데이터를 주고받기
- 간단한 웹 서버 만들기
- 카카오톡 챗봇과 Python 서버 연결하기
- GitHub를 이용해 네 명이 코드를 공유하기
- VS Code에서 코드 작성하고 실행하기
- 각자 개발한 코드를 테스트하기
- 네 명의 코드를 하나의 프로그램으로 통합하기
- 실제 사용자가 프로그램을 사용해보게 하기

AI는 모르는 내용을 설명받거나 오류를 해결하는 **개발 도구 중 하나로** 사용한다.

프로젝트의 중심은 AI 자체가 아니라 **우리가 직접 만드는 급식 알리미 서비스**이다.

---

## 3. 프로젝트 성공 기준

다음 네 가지가 이루어지면 프로젝트는 성공한 것으로 본다.

### 1. 실제 서비스가 동작한다

카카오톡에서

오늘 급식

을 입력했을 때 신봉고 급식이 표시된다.

### 2. 네 명 모두 자기 담당 코드가 있다

각 학생이 자신이 담당한 Python 파일과 함수를 직접 수정하고 완성한다.

### 3. 자기 코드를 설명할 수 있다

각자 다음 질문에 답할 수 있어야 한다.

- 이 함수에는 무엇이 들어오나요?
- 함수 안에서 무엇을 하나요?
- 결과로 무엇이 나오나요?
- 중요한 `if` 나 `for` 문은 어떤 역할을 하나요?

### 4. 네 명의 코드를 실제로 연결한다

각자 만든 프로그램이 따로 동작하는 것에서 끝나지 않고 최종적으로 하나의 챗봇 서비스로 연결되어야 한다.

---

## 4. 이번 프로젝트의 범위

### 반드시 구현할 기능

#### 기능 1

신봉고의 특정 날짜 급식을 NEIS에서 가져온다.

#### 기능 2

급식 데이터를 사람이 읽기 좋은 문장으로 만든다.

#### 기능 3

카카오톡의 질문을 받아 처리한다.

#### 기능 4

Python 웹 서버를 통해 카카오톡과 연결한다.

#### 기능 5

사용자가 `오늘 급식` 을 입력하면 실제 오늘 급식을 반환한다.

---

## 5. 시간이 남으면 추가할 기능

기본 기능이 완성된 후에만 추가한다.

### 추가 기능 1

내일 급식

### 추가 기능 2

이번 주 급식

### 추가 기능 3

급식이 없는 날 안내

오늘은 급식이 없습니다.

### 추가 기능 4

메뉴에 따라 간단한 이모지 표시

 치킨  
 매운 음식

### 추가 기능 5

예외 상황에서 친절한 메시지 출력

지금은 급식 정보를 가져올 수 없습니다.  
잠시 후 다시 시도해주세요.

## 6. 이번에는 하지 않을 것

3주 안에 프로젝트를 안정적으로 완성하기 위해 다음은 기본 범위에서 제외한다.

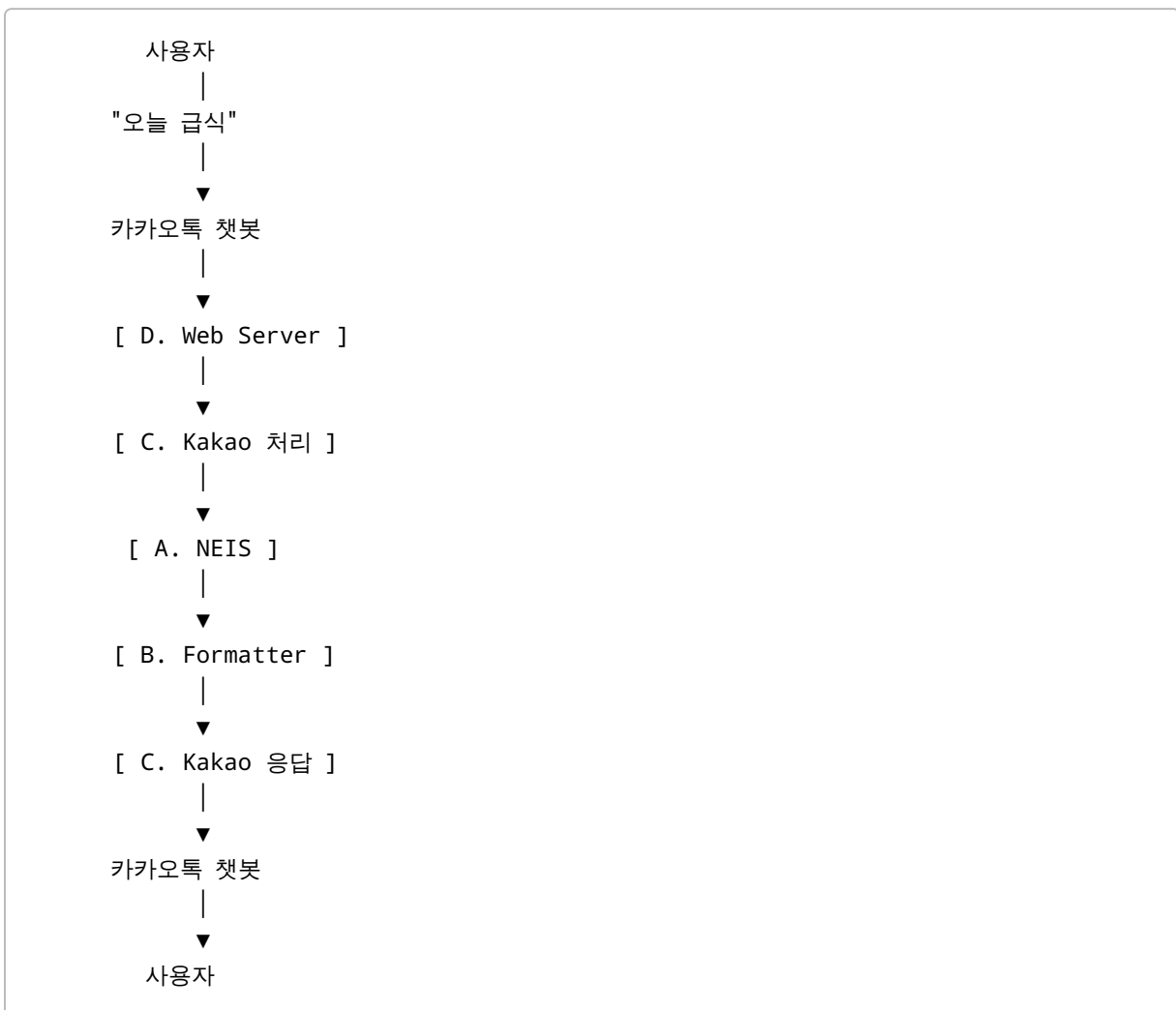
- 카카오톡 매일 아침 자동 발송
- 회원가입 시스템
- 데이터베이스
- 모바일 앱
- 별도의 웹사이트
- 로그인 기능

- 복잡한 관리자 기능
- AI 모델 학습
- 복잡한 Git branch 관리
- 복잡한 클라우드 인프라

작지만 실제로 끝까지 동작하는 서비스를 만드는 것을 우선한다.

## 7. 전체 프로그램 구조

전체 서비스는 크게 네 부분으로 나눈다.



각 학생은 이 중 하나의 부분을 주로 담당한다.

## 8. 역할 분담

코딩을 조금 경험한 학생 두 명은 A와 D를 우선 추천하고, 처음 시작하는 학생 두 명은 B와 C를 우선 추천한다.

최종 역할은 네 명이 상의해서 결정한다.

---

### 학생 A — NEIS 급식 데이터 담당

#### 담당 파일

```
lunchbot/neis.py
```

#### 목표

날짜를 입력하면 신봉고 급식 데이터를 반환한다.

예:

```
meal = get_meal("20260827")
```

결과:

```
{
  "date": "2026-08-27",
  "menu": [
    "현미밥",
    "김치찌개",
    "치킨까스",
    "깍두기"
  ],
  "calories": "735 Kcal"
}
```

#### 신봉고 정보

```
교육청 코드: J10
학교 코드: 7531013
```

## 배우게 되는 것

- 함수
- API
- requests
- JSON
- dictionary
- list
- 문자열 처리

## 혼자 테스트하는 방법

```
meal = get_meal("20260827")
print(meal)
```

다른 학생의 코드가 없어도 혼자 실행할 수 있어야 한다.

# 9. 학생 B — 급식 메시지 담당

## 담당 파일

```
lunchbot/formatter.py
```

## 목표

NEIS에서 받은 데이터를 사람이 보기 좋은 메시지로 만든다.

입력:

```
meal = {
    "date": "2026-08-27",
    "menu": [
        "현미밥",
        "김치찌개",
        "치킨까스"
    ],
    "calories": "735 Kcal"
}
```

함수:

```
message = format_meal(meal)
```

결과:

🍲 8월 27일 신봉고 급식

현미밥  
김치찌개  
치킨까스

735 Kcal

### 배우게 되는 것

- 변수
- 문자열
- list
- for
- if
- f-string
- 함수

### 혼자 테스트하는 방법

실제 NEIS가 없어도 Sample Data를 사용한다.

```
message = format_meal(sample_meal)
print(message)
```

---

## 10. 학생 C — 카카오 챗봇 데이터 담당

### 담당 파일

```
lunchbot/kakao.py
```

### 목표 1

사용자가 입력한 내용을 이해한다.

예:



```
parse_command("오늘 급식")
```

결과:

```
today
```

시간이 남으면:

```
parse_command("내일 급식")
```

결과:

```
tomorrow
```

도 구현한다.

## 목표 2

Python 문자열을 카카오 챗봇이 요구하는 JSON 형식으로 변환한다.

예:

```
make_kakao_response("오늘 급식은 돈까스입니다.")
```

## 배우게 되는 것

- if
- 함수
- dictionary
- JSON
- 입력과 출력

## 혼자 테스트하는 방법

```
command = parse_command("오늘 급식")  
print(command)
```

```
response = make_kakao_response("테스트 메시지")  
print(response)
```

---

## 11. 학생 D — 웹 서버 및 통합 담당

### 담당 파일

```
app.py
lunchbot/service.py
```

### 목표

카카오톡에서 인터넷을 통해 요청할 수 있는 Python 웹 서버를 만든다.

예:

```
POST /kakao
```

요청이 들어오면 각 학생이 만든 함수를 연결한다.

최종적으로는 다음과 같은 흐름이 된다.

```
meal = get_meal(date)
message = format_meal(meal)
response = make_kakao_response(message)

return response
```

### 배우게 되는 것

- HTTP
- POST
- 웹 서버
- API endpoint
- Flask 또는 FastAPI
- 여러 Python 파일 연결하기
- 배포

### 혼자 테스트하는 방법

처음에는 다른 학생들의 함수 대신 가짜 데이터를 사용한다.

예:

```
def get_meal(date):  
    return {  
        "date": date,  
        "menu": ["돈까스", "김치"],  
        "calories": "700 Kcal"  
    }
```

먼저

HTTP 요청 → HTTP 응답

이 정상적으로 동작하게 한다.

## 12. 네 명이 서로 기다리지 않도록 하는 방법

프로젝트에서 중요한 원칙이다.

A가 아직 NEIS 코드를 완성하지 않았다고 해서 B가 기다리면 안 된다.

따라서 미리 Sample Data를 준비한다.

예:

```
{  
    "date": "2026-08-27",  
    "menu": [  
        "현미밥",  
        "김치찌개",  
        "치킨까스",  
        "깍두기"  
    ],  
    "calories": "735 Kcal"  
}
```

각 학생은 이 데이터를 사용해 자기 프로그램을 먼저 완성한다.

즉,

A가 끝나야 B 시작  
B가 끝나야 C 시작

이 아니라,

A ┌  
B ┌  
C ┌ → 나중에 통합  
D ┌

형태로 동시에 개발한다.

## 13. 가장 먼저 정해야 할 약속

각 프로그램 사이에서 어떤 데이터를 주고받을지 미리 정한다.

이 약속은 특별한 이유가 없다면 프로젝트 중간에 변경하지 않는다.

### A → B

```
get_meal(date)
```

입력:

```
"20260827"
```

출력:

```
{  
  "date": "2026-08-27",  
  "menu": ["현미밥", "김치찌개"],  
  "calories": "735 Kcal"  
}
```

## B → C/D

```
format_meal(meal)
```

입력:

```
dict
```

출력:

```
str
```

## C

```
make_kakao_response(message)
```

입력:

```
str
```

출력:

```
dict
```

이처럼 각자의 입력과 출력이 명확해야 서로 독립적으로 작업할 수 있다.

## 14. 프로젝트 폴더

Coding Agent는 처음에 다음 구조를 준비한다.

```
shinbong-lunch-bot/  
|  
├─ app.py  
|  
└─ lunchbot/
```

```
├── __init__.py
├── neis.py
├── formatter.py
├── kakao.py
├── service.py
├── tests/
│   ├── test_neis.py
│   ├── test_formatter.py
│   ├── test_kakao.py
│   └── test_service.py
├── samples/
│   ├── meal.json
│   └── kakao_request.json
├── docs/
│   └── architecture.md
├── .env.example
├── .gitignore
├── requirements.txt
└── README.md
```

## 15. Coding Agent가 미리 할 일

Coding Agent는 프로그램을 완성하지 않는다.

학생들이 바로 코드 작업을 시작할 수 있도록 다음만 준비한다.

## 준비할 것

- 폴더 구조
- Python 파일
- 함수 이름
- 함수 설명
- 입력/출력 예시
- Sample Data
- 간단한 테스트 코드
- 필요한 Python package
- `.gitignore`
- `.env.example`
- 초보자용 README

- 프로그램 구조 설명

## 하지 않을 것

학생들이 맡은 함수 내부의 핵심 구현을 미리 완성하지 않는다.

예:

```
def get_meal(date: str) -> dict:
    """
    특정 날짜의 신봉고 급식 정보를 가져온다.
    """

    # TODO: Student A
    raise NotImplementedError
```

이처럼 학생이 작업해야 할 곳을 명확하게 남겨둔다.

---

# 16. 개발 도구

## Python

실제 프로그램을 작성하는 언어.

## VS Code

코드를 작성하고 실행하기 위한 프로그램.

## GitHub

네 명이 같은 프로젝트 코드를 공유하는 장소.

## Git

GitHub와 내 컴퓨터 사이에서 코드를 주고받는 도구.

## AI 도구

모르는 내용을 설명받거나 오류를 해결하고, 작은 코드 작성에 도움을 받을 때 사용한다.

---

## 17. 처음 VS Code를 사용할 때 알아야 할 것

VS Code의 모든 기능을 배울 필요는 없다.

다음만 알면 프로젝트를 진행할 수 있다.

### 프로젝트 폴더 열기

```
File → Open Folder
```

### Python 파일 열기

예:

```
lunchbot/formatter.py
```

### Terminal 열기

```
Terminal → New Terminal
```

### Python 실행

```
python filename.py
```

### 테스트 실행

```
pytest
```

### GitHub 작업

왼쪽의 **Source Control** 메뉴를 사용한다.

Git 명령어를 전부 외울 필요는 없다.

---

## 18. GitHub 사용 방법

이번 프로젝트에서는 Git을 복잡하게 사용하지 않는다.



Branch와 Pull Request는 기본 과정에서 사용하지 않는다.

항상 다음 순서만 기억한다.

- ① Pull
- ② 내 코드 수정
- ③ 실행해서 확인
- ④ Commit
- ⑤ Push

---

## 19. GitHub에서 알아야 할 네 단어

### Clone

GitHub에 있는 프로젝트를 처음으로 내 컴퓨터에 가져오는 것.

### Pull

친구들이 올린 최신 코드를 내 컴퓨터로 가져오는 것.

### Commit

내가 수정한 내용을 하나의 작업 기록으로 저장하는 것.

### Push

내 Commit을 GitHub로 올리는 것.

---

## 20. GitHub 작업 규칙

### 규칙 1

코딩을 시작하기 전에 항상 **Pull**한다.

Pull → 수정 → 테스트 → Commit → Push

## 규칙 2

가능하면 자기 담당 파일만 수정한다.

```
A → neis.py
B → formatter.py
C → kakao.py
D → app.py / service.py
```

## 규칙 3

공용 파일을 여러 명이 동시에 수정하지 않는다.

예:

```
README.md
requirements.txt
```

이런 파일을 수정할 때는 다른 친구들에게 먼저 알린다.

## 규칙 4

Git conflict가 발생하면 혼자 억지로 해결하지 않는다.

팀원과 함께 확인한다.

# 21. Commit 메시지

Commit에는 무엇을 했는지 간단하게 적는다.

좋은 예:

```
신봉고 급식 API 요청 추가

급식 메뉴 출력 형식 추가

오늘 급식 명령 처리 추가
```

카카오 응답 JSON 생성 기능 추가

급식 없는 날 처리

다음과 같은 메시지는 피한다.

```
update
fix
aaa
final
```

Commit 기록은 나중에 각자 어떤 일을 했는지 확인할 수 있는 프로젝트 기록이 된다.

## 22. AI 사용 원칙

AI를 사용해도 된다.

하지만 AI에게 프로젝트 전체를 대신 만들어달라고 하지 않는다.

좋은 사용 방법:

이 코드에서 for문이 어떤 역할을 하는지 설명해줘.

이 오류가 발생하는 이유를 Python 초보자에게 설명해줘.

이 함수에서 급식이 없는 경우만 처리하고 싶은데  
어디를 수정하면 되는지 알려줘.

이 코드가 어떻게 동작하는지 한 줄씩 설명해줘.

다음과 같은 사용은 피한다.

급식 챗봇 전체를 만들어줘.

내 파일을 전부 완성해줘.

중요한 기준은 하나이다.

내 코드에 들어간 내용은 내가 설명할 수 있어야 한다.

---

## 23. 테스트 방법

각 학생의 프로그램은 다른 학생 코드가 없어도 테스트할 수 있어야 한다.

Coding Agent는 기본적인 `pytest` 테스트를 준비한다.

예:

```
def test_format_meal():
    meal = {
        "date": "2026-08-27",
        "menu": ["현미밥", "김치찌개"],
        "calories": "735 Kcal"
    }

    message = format_meal(meal)

    assert "현미밥" in message
    assert "김치찌개" in message
```

학생은 코드를 작성한 뒤:

```
pytest
```

를 실행한다.

목표는:

```
코드 작성
↓
테스트
↓
실패하면 수정
↓
다시 테스트
```

↓  
성공

을 경험하는 것이다.

## 24. 비밀번호와 API Key

API Key나 token이 생기면 GitHub에 올리면 안 된다.

다음 파일을 사용한다.

```
.env
```

실제 값은 `.env`에 넣는다.

GitHub에는:

```
.env.example
```

만 올린다.

`.gitignore`에는 반드시:

```
.env
```

를 포함한다.

## 25. 개발 순서

### Step 1. 개발 환경 준비

네 명 모두 다음을 설치하고 실행한다.

- Python
- VS Code
- Git
- GitHub 계정

그리고 Repository를 Clone한다.

첫 번째 프로그램:

```
print("Hello lunch bot!")
```

을 실행한다.

그 후 한 번:

수정 → Commit → Push

까지 해본다.

---

## 26. Step 2. 자기 프로그램 혼자 만들기

학생 A:

날짜 → 급식 데이터

학생 B:

급식 데이터 → 메시지

학생 C:

사용자 입력 → 카카오 응답

학생 D:

HTTP 요청 → Python 서버 → HTTP 응답

이 단계에서는 다른 친구의 프로그램과 연결하지 않는다.

---

## 27. Step 3. 두 명씩 먼저 연결

전체를 한 번에 합치지 않는다.

먼저:

A + B

를 연결한다.

NEIS  
↓  
급식 데이터  
↓  
Formatter  
↓  
사람이 읽는 메시지

그리고:

C + D

를 연결한다.

HTTP Request  
↓  
Kakao 처리  
↓  
HTTP Response

---

## 28. Step 4. 전체 연결

두 부분이 정상적으로 동작하면 최종적으로 모두 연결한다.

카카오톡  
↓  
Web Server  
↓

사용자 명령 처리  
↓  
NEIS 급식 조회  
↓  
메시지 생성  
↓  
Kakao 응답 생성  
↓  
카카오톡

## 29. Step 5. 실제 카카오톡 연결

개발 중에는 학생 컴퓨터에서 Python 서버를 실행한다.

예:

```
localhost:8000
```

하지만 카카오는 학생 컴퓨터의 `localhost`에 직접 접근할 수 없다.

필요하면 ngrok과 같은 도구를 이용해 테스트한다.

카카오톡  
↓  
ngrok  
↓  
학생 컴퓨터  
↓  
Python 서버

이를 통해 최종 배포 전에 실제 카카오톡에서 테스트할 수 있다.

## 30. Step 6. 최종 배포

최종적으로 Python 서버는 인터넷에서 접근 가능한 HTTPS 주소에 배포한다.

예:



https://example.com/kakao

Vercel 등 간단한 클라우드 서비스를 우선 검토한다.

서비스마다 계정 연령 제한이나 이용약관이 있으므로 실제 계정을 만들기 전에 확인한다.

## 31. 오류 상황 처리

기본 기능이 완성되면 다음 정도는 처리한다.

### 급식이 없는 경우

오늘은 급식이 없습니다.

### NEIS 연결 실패

지금은 급식 정보를 가져올 수 없습니다.  
잠시 후 다시 시도해주세요.

### 모르는 질문

사용자:

안녕

응답:

'오늘 급식'이라고 입력해주세요!

## 32. 발표용 안전장치

발표 당일 인터넷 또는 NEIS에 문제가 생길 수 있다.

가능하면 실제 API가 없어도 Sample Data를 이용해 프로그램을 시연할 수 있도록 한다.

예:

```
DEMO_MODE=true
```

일 때 Sample Data를 사용하는 방식이다.

이 기능은 시간이 부족하면 생략해도 된다.

---

## 33. 3주 작업 일정

### 1주차 — 준비 + 각자 첫 기능 만들기

#### 목표

모든 학생이 자기 컴퓨터에서 프로젝트를 실행하고 자기 역할을 이해한다.

#### 해야 할 일

- Python 설치 확인
- VS Code 설치
- Git 설치
- GitHub 준비
- Repository Clone
- Python 실행
- Pull / Commit / Push 연습
- 역할 결정
- 각자 담당 파일 열어보기
- Sample Data 확인
- 자기 함수 첫 구현
- `pytest` 실행

#### 1주차 완료 기준

네 명 모두 다음을 할 수 있어야 한다.

```
프로젝트 열기
Python 실행하기
자기 파일 수정하기
테스트 실행하기
Commit 하기
Push 하기
```

---

## 34. 2주차 — 각자 기능 완성 + 부분 통합

### A

실제 NEIS API 연결.

### B

급식 메시지 완성.

### C

오늘 급식 입력과 Kakao Response 완성.

### D

웹 서버 완성.

### 이후

A + B

연결.

C + D

연결.

각자 테스트를 통과시킨다.

---

## 35. 3주차 — 전체 통합 + 실제 서비스 + 발표

### 초반

A/B/C/D 전체 연결.

### 중반

카카오톡 연결.

실제 휴대폰에서 테스트.

오류 수정.

## 후반

GitHub 정리.

시연 준비.

발표 자료 만들기.

네 명 모두 자기 부분 설명 연습.

---

## 36. 발표 구성

발표는 기술을 많이 나열하기보다 우리가 어떤 문제를 해결했고 어떻게 나누어 만들었는지를 중심으로 한다.

### 1. 문제

급식을 확인하려면 학교 홈페이지 등을 찾아봐야 한다.

카카오톡에서 바로 확인할 수 있을까?

### 2. 해결 아이디어

NEIS의 급식 데이터를 Python으로 가져와 카카오톡 챗봇으로 알려준다.

### 3. 프로그램 구조

Kakao → Python → NEIS → Python → Kakao

### 4. 역할 분담

각 학생이 자기 담당 부분을 설명한다.

### 5. 실제 시연

오늘 급식

을 입력해 실제 메뉴가 나오는 모습을 보여준다.

## 6. 통합 과정

각자 독립적으로 만든 프로그램을 어떤 방식으로 연결했는지 설명한다.

## 7. 어려웠던 점

예:

- API 데이터 형식 이해
- JSON 처리
- 카카오 응답 형식
- 서버 연결
- GitHub 사용
- 여러 파일을 합칠 때 생긴 문제

## 8. 배운 점

네 명이 각자 한 부분씩 맡고 하나의 프로그램으로 합쳐본 경험을 설명한다.

---

# 37. 각 학생이 발표할 내용

### A

저는 NEIS API에서 신봉고의 급식 정보를 가져오는 부분을 담당했습니다.

날짜와 학교 정보를 어떻게 보내고 어떤 데이터를 받았는지 설명한다.

### B

저는 받아온 급식 데이터를 사용자가 읽기 좋은 메시지로 바꾸는 부분을 담당했습니다.

`list`, `for`, 문자열 등을 설명한다.

### C

저는 사용자의 질문을 처리하고 카카오톡이 이해하는 응답 형식을 만드는 부분을 담당했습니다.

입력과 JSON 응답을 설명한다.

## D

저는 각 프로그램을 웹 서버로 연결하고 카카오톡에서 접근할 수 있도록 하는 부분을 담당했습니다.

HTTP request와 전체 연결 과정을 설명한다.

---

## 38. 프로젝트에서 가장 중요한 원칙

### 첫 번째

각자 자기 프로그램을 갖는다.

### 두 번째

각자의 프로그램은 혼자 테스트할 수 있다.

### 세 번째

다른 친구의 프로그램이 끝날 때까지 기다리지 않는다.

### 네 번째

마지막에는 반드시 네 명의 코드를 하나로 연결한다.

### 다섯 번째

모르는 것은 AI나 인터넷을 이용해서 배워도 되지만, 자기 코드가 어떻게 동작하는지는 설명할 수 있어야 한다.

---

## Appendix — Coding Agent용 Skeleton 작성 지침

이 부분은 초기 Repository를 만드는 Coding Agent에게 전달한다.

### 목적

고등학교 1학년 Python 초보 학생 네 명이 직접 완성할 수 있는 프로젝트 Skeleton을 만든다.

중요한 기준은:

완성된 프로그램을 만들어주는 것이 아니라 학생들이 바로 코딩을 시작할 수 있는 환경을 만들어주는 것

이다.

---

## 1. Repository 구조

다음 구조를 생성한다.

```
shinbong-lunch-bot/
├── app.py
├── lunchbot/
│   ├── __init__.py
│   ├── neis.py
│   ├── formatter.py
│   ├── kakao.py
│   └── service.py
├── tests/
│   ├── test_neis.py
│   ├── test_formatter.py
│   ├── test_kakao.py
│   └── test_service.py
├── samples/
│   ├── meal.json
│   └── kakao_request.json
├── docs/
│   └── architecture.md
├── .env.example
├── .gitignore
├── requirements.txt
└── README.md
```

---

## 2. 학생이 구현할 함수

### Student A

```
def get_meal(date: str) -> dict:
    """
    특정 날짜의 신봉고 급식 정보를 반환한다.
```

```
Example input:  
    "20260827"
```

```
Example output:  
    {  
        "date": "2026-08-27",  
        "menu": ["현미밥", "김치찌개"],  
        "calories": "735 Kcal"  
    }  
    ""
```

```
# TODO: Student A  
raise NotImplementedError
```

---

## Student B

```
def format_meal(meal: dict) -> str:  
    """  
    급식 데이터를 사람이 읽기 좋은 문자열로 변환한다.  
    """  
  
    # TODO: Student B  
    raise NotImplementedError
```

---

## Student C

```
def parse_command(text: str) -> str:  
    """  
    '오늘 급식' 등의 사용자 입력을  
    내부 command로 변환한다.  
    """  
  
    # TODO: Student C  
    raise NotImplementedError
```

```
def make_kakao_response(message: str) -> dict:  
    """  
    일반 문자열을 Kakao chatbot response JSON으로 변환한다.  
    """
```



```
# TODO: Student C
raise NotImplementedError
```

## Student D

POST /kakao endpoint의 기본 구조만 생성한다.

학생이 HTTP 요청을 처리하는 부분을 이해하고 수정할 수 있도록 단순한 형태로 만든다.

service.py 와 연결될 수 있도록 구조만 준비한다.

## 3. 통합 함수

```
def handle_message(text: str):
    # 1. 사용자 입력 해석
    # 2. 날짜 결정
    # 3. 급식 가져오기
    # 4. 메시지 만들기
    # 5. Kakao response 만들기

    # TODO: Integration
    raise NotImplementedError
```

불필요한 class나 architecture를 추가하지 않는다.

## 4. 코드 난이도

가능하면 다음만 이용한다.

```
function
if
for
list
dict
string
requests
simple HTTP endpoint
```

다음은 사용하지 않거나 꼭 필요한 경우에만 사용한다.

- 복잡한 class 구조
- async programming
- Docker
- database
- ORM
- dependency injection
- advanced typing
- decorator 중심 architecture
- 복잡한 configuration system

학생이 읽고 이해할 수 있는 단순한 코드가 가장 중요하다.

---

## 5. Sample Data

학생들이 서로 기다리지 않고 개발할 수 있도록 충분한 Sample Data를 제공한다.

예:

```
{
  "date": "2026-08-27",
  "menu": [
    "현미밥",
    "김치찌개",
    "치킨까스",
    "깍두기"
  ],
  "calories": "735 Kcal"
}
```

Kakao 요청도 실제 구조를 참고한 Sample JSON을 제공한다.

---

## 6. Tests

각 학생의 함수를 독립적으로 실행할 수 있는 간단한 `pytest` 를 준비한다.

테스트는 구현 방법을 강제하지 않는다.

다음만 확인하는 수준으로 만든다.

올바른 입력을 받는가?  
필요한 결과를 반환하는가?  
반환 데이터 형식이 맞는가?

학생이 테스트 실패 메시지를 보고 수정할 수 있을 정도로 단순하게 작성한다.

## 7. README

README는 Python과 VS Code를 처음 사용하는 학생이 위에서부터 그대로 따라갈 수 있어야 한다.

최소한 다음 내용을 포함한다.

1. 필요한 프로그램 설치 확인
2. Repository Clone
3. VS Code에서 폴더 열기
4. Python virtual environment 만들기
5. package 설치
6. 프로그램 실행
7. pytest 실행
8. GitHub Pull
9. 코드 수정
10. Commit
11. Push

Branch와 Pull Request는 기본 설명에서 제외한다.

## 8. NEIS 설정

다음 신봉고 값을 한곳에서 관리하도록 한다.

```
ATPT_OFCDC_SC_CODE = J10  
SD_SCHUL_CODE = 7531013
```

필요한 API URL이나 설정도 가능한 한 눈에 찾기 쉬운 위치에 둔다.

## 9. Secret 관리

`.env` 는 Git에 포함하지 않는다.

`.gitignore`에 `.env`를 추가한다.

`.env.example`에는 실제 secret 없이 필요한 환경변수 이름만 작성한다.

---

## 10. Documentation

`docs/architecture.md`에는 전체 흐름을 매우 간단하게 설명한다.

```
Kakao
  ↓
Web Server
  ↓
Command Parser
  ↓
NEIS
  ↓
Formatter
  ↓
Kakao Response
```

그리고 각 함수의 입력과 출력 형식을 기록한다.

별도의 AI 사용 기록 문서는 만들지 않는다.

---

## 11. Coding Agent의 최우선 원칙

이 프로젝트는 고등학교 1학년 학생 네 명이 직접 완성하기 위한 교육용 프로젝트이다.

따라서:

학생이 작성해야 할 핵심 코드를 대신 구현하지 않는다.

학생들이 무엇을 해야 할지 몰라 막히지 않도록 프로젝트 구조와 실행 환경은 충분히 준비한다.

각 학생의 담당 영역은 다른 학생의 코드가 없어도 Sample Data로 독립적으로 테스트할 수 있어야 한다.

복잡하고 세련된 구현보다 단순하고 이해하기 쉬운 구현을 선택한다.

모든 설계에서 “학생이 이 코드를 직접 설명할 수 있는가?”를 가장 중요한 기준으로 사용한다.

최종 목표는 **완성도 높은 코드를 AI가 만들어주는 것이 아니라,**

네 명의 학생이 각자 작은 프로그램을 직접 완성하고 그것을 하나의 실제 서비스로 연결해보는 것이다.